

AulaNube: arquitectura de microservicios en la nube para la plataforma de capacitación corporativa de CapacitaPro S.A.S.

[Número de grupo]

Augusto César Cañola Ortiz

[Nombre del integrante]

[Nombre del integrante]

[Nombre del integrante]

Ingeniería de Software, Tecnológica del Oriente

Arquitectura de Software (20262 – N5, Grupo A)

Docente: [Nombre del docente]

Octubre de 2026

Resumen

Este informe presenta la arquitectura propuesta para CapacitaPro S.A.S., empresa ficticia que ofrece capacitación virtual obligatoria a 38 empresas cliente con una instalación de WordPress por cliente, modelo que colapsa en los cierres de campaña, hace esperar al usuario mientras se genera el certificado y multiplica el mantenimiento. A partir del análisis de requisitos se formularon diez escenarios de calidad medibles que orientaron el diseño de AulaNube, una plataforma multiempresa basada en microservicios, con un API Gateway, una base de datos por servicio y eventos asíncronos en Redis Streams. La arquitectura se documentó con diagramas UML de casos de uso, clases, secuencia y despliegue, y se justificaron patrones de diseño para la nube como Transactional Outbox, Queue-Based Load Leveling, Competing Consumers y Circuit Breaker. El prototipo se desplegó en Oracle Cloud Infrastructure, sobre un clúster K3s de dos nodos Ampere A1 de la capa gratuita, y las mediciones confirmaron los escenarios: una campaña masiva responde en menos de 300 ms, el gateway responde en 110 ms con un servicio caído y ningún certificado se pierde cuando falla el trabajador.

Palabras clave: arquitectura de software, microservicios, patrones de diseño, computación en la nube, UML.

Tabla de contenido

Resumen 2
AulaNube: arquitectura de microservicios en la nube para la plataforma de capacitación corporativa de CapacitaPro S.A.S. 4
Caso de estudio: CapacitaPro S.A.S. 4
Análisis de requisitos 5
Arquitectura propuesta 7
Estilo arquitectónico 7
Relación con los principios de la unidad 8
Diseño de la arquitectura mediante UML 9
Diagrama de casos de uso 10
Diagrama de clases 11
Diagrama de secuencia 12
Diagrama de despliegue 13
Selección y justificación de patrones de diseño 14
Prototipo funcional en la nube 15
Tecnologías y despliegue 15
Evidencias de los patrones 15
Conclusiones 18
Referencias 19

AulaNube: arquitectura de microservicios en la nube para la plataforma de capacitación corporativa de CapacitaPro S.A.S.

La arquitectura de software es la organización fundamental de un sistema, expresada en sus componentes, sus relaciones y los principios que guían su diseño y evolución (Bass et al., 2012). Como estas decisiones se toman temprano y son costosas de revertir, deben responder a los atributos de calidad que el negocio necesita y no solo a la funcionalidad (Tecnológica del Oriente, 2026).

El informe presenta la solución del equipo para CapacitaPro S.A.S. Primero describe el caso y sus requisitos; luego la arquitectura propuesta, su modelado en UML y los patrones de diseño seleccionados; por último, el prototipo desplegado en la nube y las pruebas que comprueban cada decisión. El código fuente, los diagramas y las instrucciones de despliegue están en el repositorio https://github.com/augustocco/aulanube, y la aplicación funciona en https://aulanube.149-130-191-139.sslip.io.

Caso de estudio: CapacitaPro S.A.S.

CapacitaPro S.A.S. es una empresa ficticia de Bucaramanga que diseña y administra cursos virtuales que la ley o los contratos hacen obligatorios: inducción al Sistema de Gestión de Seguridad y Salud en el Trabajo (SG-SST), trabajo seguro en alturas, protección de datos personales y atención humanizada en salud. Atiende a 38 empresas cliente y unos 14.000 usuarios. Cada cliente recibe su propia instalación de WordPress con un plugin de LMS; hoy hay 38 instalaciones en cinco servidores. La Tabla 1 resume los problemas de ese modelo.

Tabla 1

Problemas de la operación actual de CapacitaPro

Problema	Evidencia
Picos en los cierres de campaña	El cierre de una campaña de 1.100 trabajadores dejó el servidor sin respuesta 40 minutos y afectó a otros siete clientes alojados en la misma máquina.
Certificado generado dentro de la solicitud	Respuestas de 8 a 25 s al terminar un curso; si el usuario cierra la página, el certificado no se genera.
Correo que bloquea la operación	Cursos terminados que no quedan registrados porque el servidor de correo tardó en responder.
Mantenimiento multiplicado	Cada actualización se aplica 38 veces y ocupa cerca del 40 % del tiempo del equipo técnico.
Sin verificación de certificados	Auditores y contratantes deben pedir confirmación por correo, lo que tarda días.

Nota. Elaboración propia a partir del caso de estudio.

La gerencia pide una sola plataforma multiempresa en la nube en la que cada cliente vea solo su información; que el cierre de campaña de un cliente no afecte a los demás; certificados verificables por terceros; que la caída de una parte no detenga el resto; crecer por partes; bajo costo durante el piloto y cumplimiento de la Ley 1581 de 2012 (Congreso de la República de Colombia, 2012). El prototipo cubre el flujo central: ingreso, catálogo por empresa, avance de lecciones, emisión del certificado, avisos, verificación pública, gestión de la empresa y un panel de observación de la arquitectura.

Análisis de requisitos

Se aplicaron cuatro técnicas: análisis documental del caso y de sus incidentes; una entrevista simulada en la que el equipo representó a la gerencia, a una administradora de empresa cliente y a un colaborador; historias de usuario, priorizadas con el método MoSCoW (debe, debería, podría), y escenarios de atributos de calidad, que expresan cada requisito no funcional con un estímulo, un entorno y una medida de respuesta verificable (Bass et al., 2012). Las Tablas 2 y 3 presentan el resultado.

Tabla 2

Requisitos funcionales

ID	Historia de usuario	Prioridad
RF01	Como usuario quiero iniciar sesión para acceder a la plataforma de mi empresa.	Debe
RF02	Como estudiante quiero ver solo los cursos de mi empresa, con mi avance.	Debe
RF03	Como estudiante quiero inscribirme en un curso y marcar lecciones como vistas.	Debe
RF04	Como estudiante quiero que el curso quede completado al ver la última lección.	Debe
RF05	Como estudiante quiero un certificado PDF con código único al completar un curso.	Debe
RF06	Como estudiante quiero un aviso al completar el curso y cuando el certificado esté listo.	Debe
RF07	Como verificador externo quiero consultar un certificado por su código, sin cuenta.	Debe
RF08	Como administrador quiero crear cursos y registrar colaboradores de mi empresa.	Debe
RF09	Como administrador quiero ver inscritos, completados y certificados de mi empresa.	Debe
RF10	Como administrador quiero cerrar una campaña masiva sin degradar la plataforma.	Debe
RF11	Como equipo técnico quiero ver en vivo la salud de los servicios y la cola de eventos.	Debería
RF12	Como administrador quiero cargar colaboradores desde un archivo.	Podría

Nota. Cada requisito se traza a un caso de uso del diagrama de la Figura 2. RF12 queda fuera del prototipo.

Tabla 3

Requisitos no funcionales expresados como escenarios de calidad

ID	Atributo	Escenario y medida de aceptación
RNF01	Escalabilidad	150 colaboradores terminan el curso en el mismo minuto: la respuesta tarda menos de 1 s y todos los certificados se emiten.
RNF02	Rendimiento	Al completar la última lección la respuesta tarda menos de 500 ms y el certificado está disponible en menos de 10 s.
RNF03	Disponibilidad	Si el servicio de certificados cae, los demás siguen respondiendo y el gateway contesta en menos de 300 ms.
RNF04	Confiabilidad	Si Redis o un trabajador fallan, ningún certificado se pierde y un evento repetido no genera duplicados.
RNF05	Seguridad y privacidad	Un usuario de una empresa no puede leer datos de otra ni falsificar su rol; los servicios internos no son accesibles desde fuera.
RNF06	Equidad entre clientes	Una empresa que supera 20 solicitudes por segundo recibe el código 429 sin afectar a las demás.
RNF07	Modificabilidad	Cambiar el envío de avisos por un correo real solo modifica el servicio de notificaciones.
RNF08	Desplegabilidad	Una versión nueva de un servicio se despliega sin detener los demás.
RNF09	Costo	La infraestructura del piloto se mantiene dentro de la capa gratuita del proveedor de nube.
RNF10	Observabilidad	Una falla se puede seguir con el identificador de correlación y el panel de estado.

Nota. Elaboración propia con el formato de escenarios de calidad de Bass et al. (2012).

Arquitectura propuesta

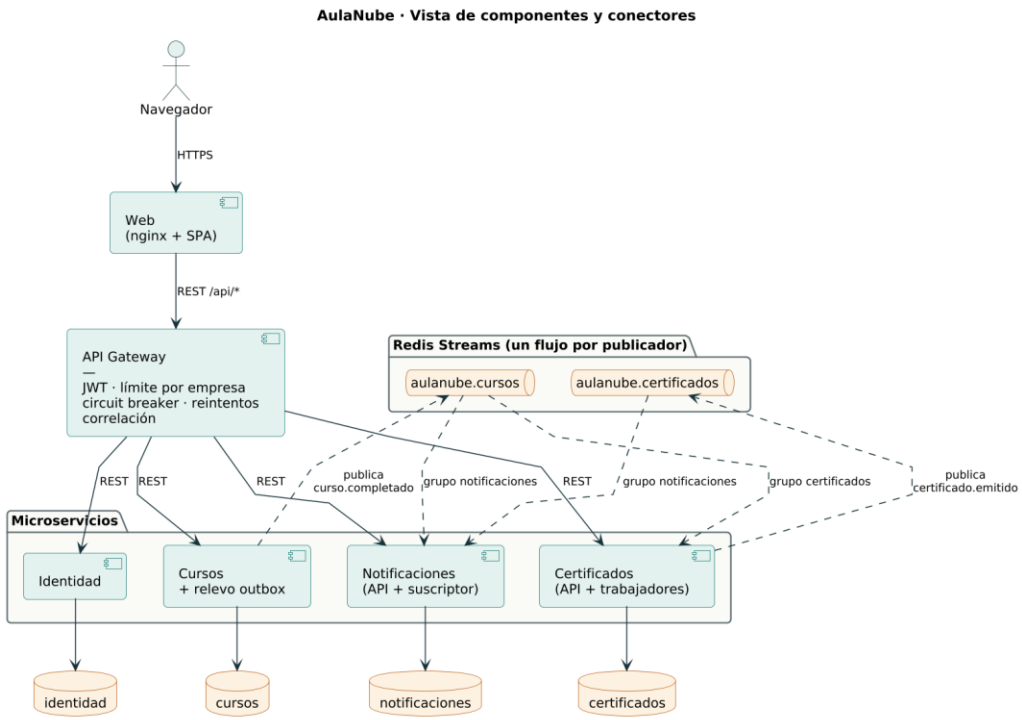
Estilo arquitectónico

Se eligió una arquitectura de microservicios con comunicación híbrida: REST síncrono para lo que el usuario consulta y necesita ver de inmediato, y eventos asíncronos para lo que puede ocurrir después, como emitir el certificado y enviar el aviso. Los escenarios RNF01, RNF03 y RNF08 exigen escalar y aislar fallos por partes. Un monolito modular habría sido más sencillo de operar, pero escala y falla como una sola unidad, que es precisamente el problema de CapacitaPro (Red Hat, 2023; Redwerk, 2025).

La Figura 1 muestra los componentes. La interfaz web envía todas las solicitudes al API Gateway, única entrada al sistema. Detrás están cuatro servicios, cada uno con su propia base de datos: identidad (empresas, usuarios y tokens), cursos (catálogo, inscripciones y progreso), certificados (generación del PDF y verificación pública) y notificaciones (avisos y correo simulado).

Figura 1

Vista de componentes y conectores de AulaNube



Nota. Elaboración propia. Las flechas continuas son llamadas REST; las punteadas, eventos en Redis Streams.

Relación con los principios de la unidad

La propuesta aplica los principios de modularidad estudiados. Hay **alta cohesión** porque cada servicio agrupa un solo concepto del dominio: certificados no calcula progreso y cursos no genera PDF. Hay **bajo acoplamiento** porque los servicios no se llaman entre sí; cursos publica el evento `curso.completado` sin saber quién lo consume, de modo que agregar un nuevo interesado, como facturación, no exige modificarlo. Y se respeta el **ocultamiento de información** propuesto por Parnas (1972): la base de datos de cada servicio es un detalle interno, por lo que trasladar los PDF a un almacenamiento de objetos no afectaría a ningún otro módulo mientras se conserven la API y los eventos.

Tabla 4

Decisiones arquitectónicas y sus compromisos

Decisión	Beneficio	Costo aceptado
Certificados y avisos asíncronos	El usuario no espera por el PDF ni por el correo; los picos se absorben en la cola.	Consistencia eventual: el certificado aparece segundos después.
Una base por servicio en una instancia de PostgreSQL	Aislamiento lógico y permisos separados sin el costo de cuatro servidores.	La instancia es un punto compartido; en producción se separaría.
Redis Streams en lugar de un bróker dedicado	Grupos de consumidores, confirmación y reclamo de pendientes con poca memoria.	Menor durabilidad que Kafka; se mitiga con persistencia AOF y la bandeja de salida.
Código común copiado en cada servicio	Ningún servicio depende de las versiones de una librería compartida.	Duplicación de unas 150 líneas.

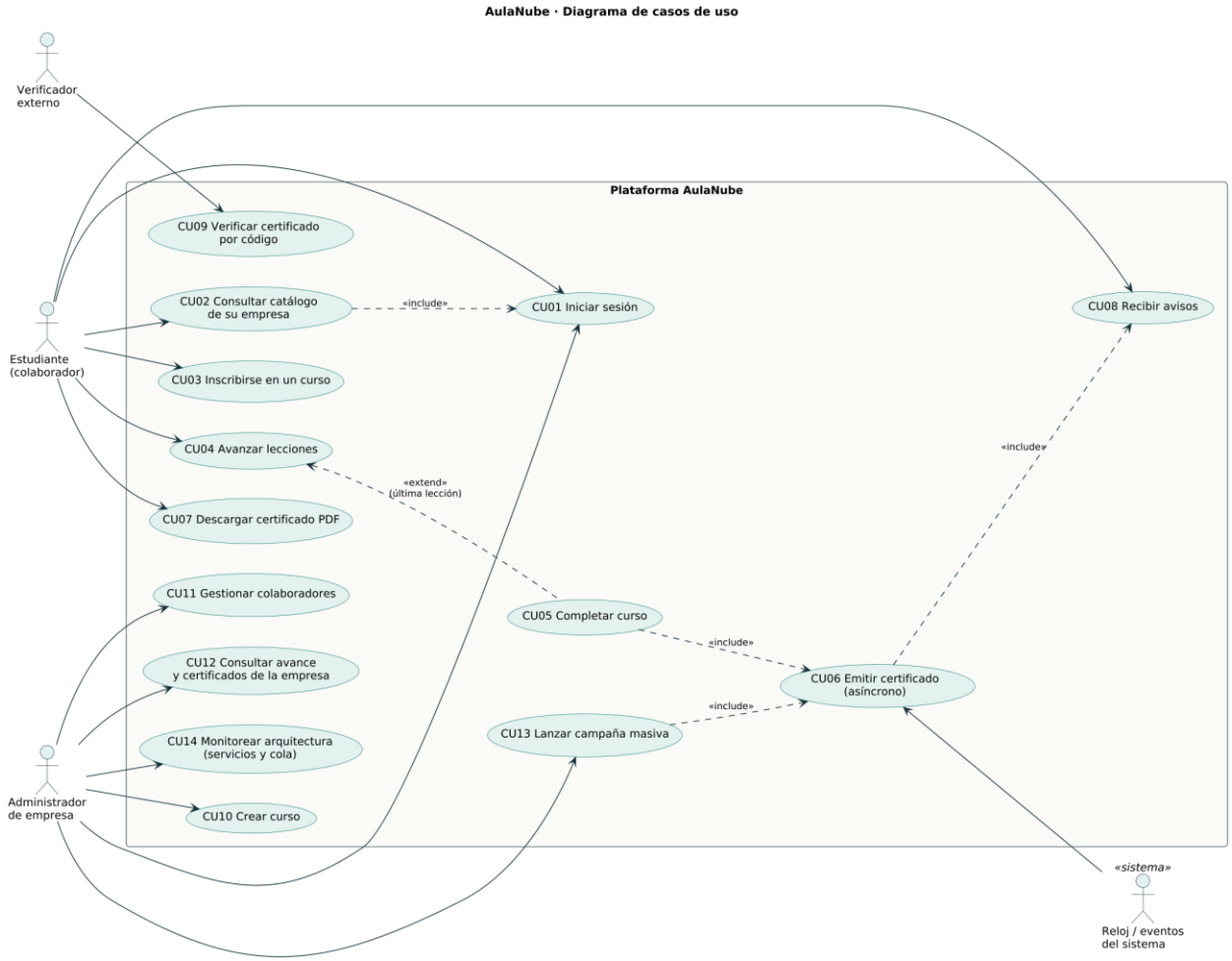
Diseño de la arquitectura mediante UML

Los diagramas se elaboraron en PlantUML siguiendo la especificación UML 2.5.1 (Object Management Group, 2017). Su código fuente está en la carpeta `docs/uml` del repositorio, de modo que evolucionan junto con el código.

Diagrama de casos de uso

Figura 2

Diagrama de casos de uso de AulaNube



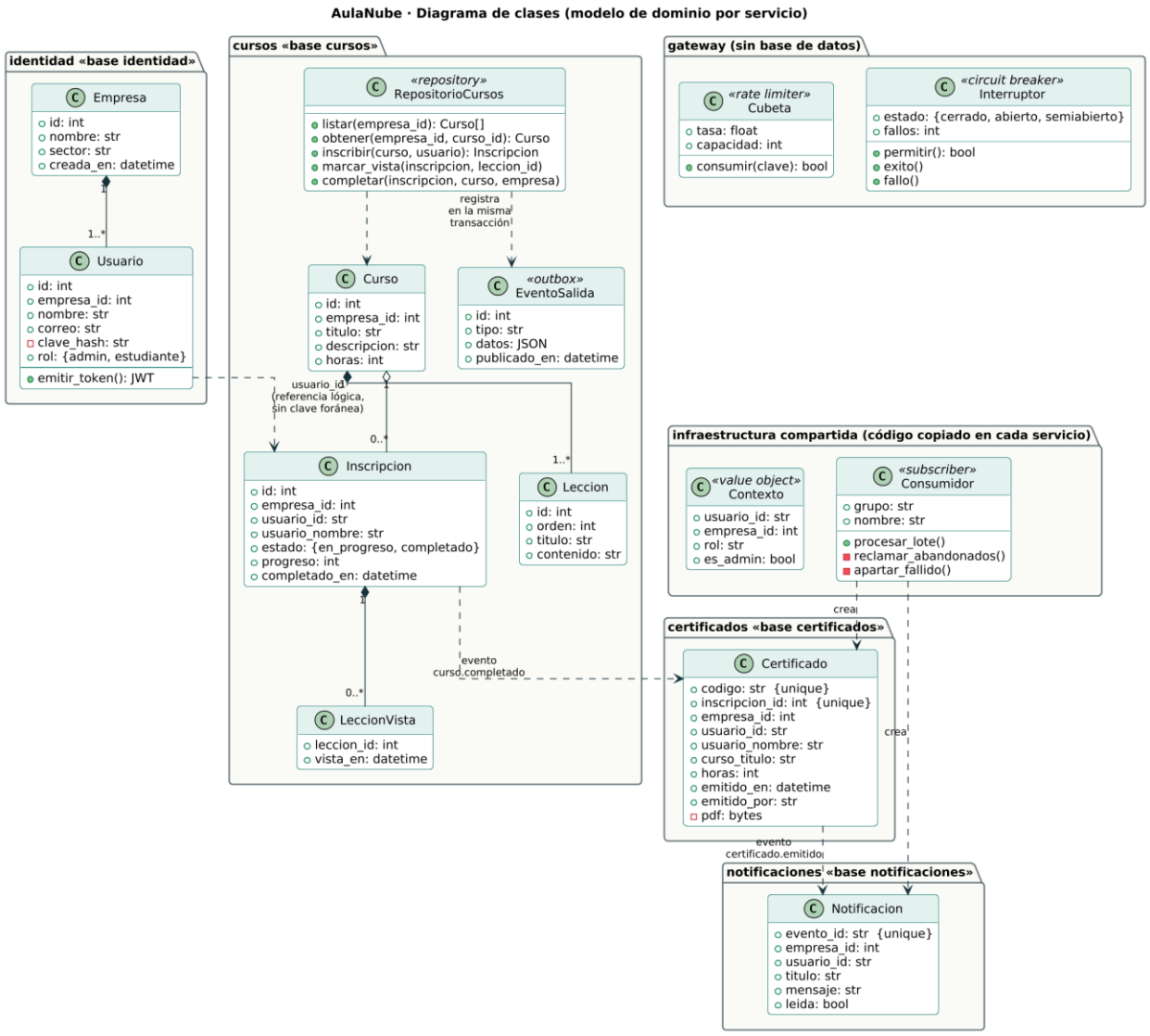
Nota. Elaboración propia.

Cuatro actores interactúan con la plataforma: el estudiante, el administrador de la empresa cliente, un verificador externo sin cuenta y el propio sistema. «Completar curso» extiende el avance de lecciones al marcar la última e incluye «Emitir certificado», que a su vez incluye «Recibir avisos». El diagrama es relevante para la arquitectura porque muestra que la emisión la dispara un evento y no una persona, lo que la señala como asíncrona, y porque el verificador sin cuenta obliga a una ruta pública controlada en el gateway.

Diagrama de clases

Figura 3

Modelo de dominio por servicio



Nota. Elaboración propia. Cada paquete corresponde a un servicio y a su base de datos.

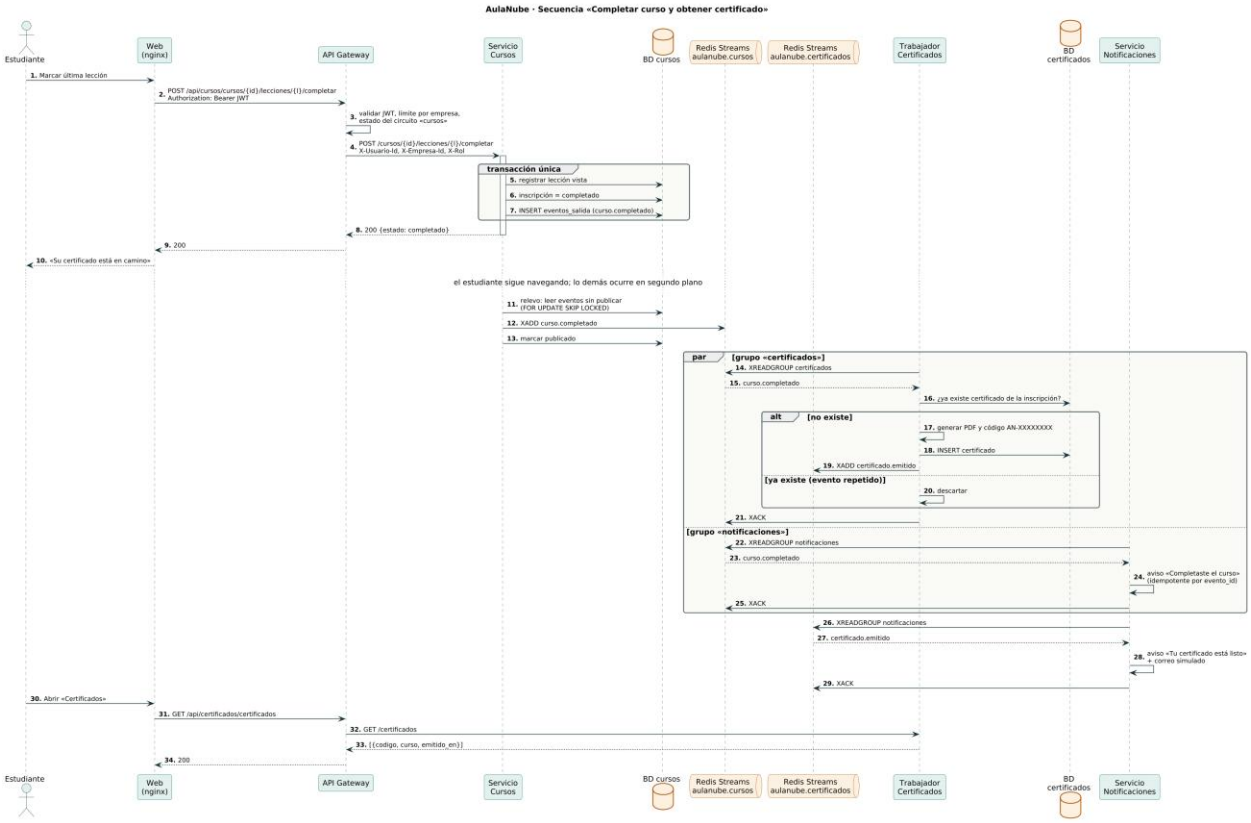
Cada paquete agrupa las clases de un servicio y ninguna asociación lo cruza: entre servicios solo hay referencias lógicas, como `usuario_id`, o eventos. Esto comprueba la decisión de una base de datos por servicio. `RepositorioCursos` encapsula el acceso a datos e impone el

filtro por empresa, y las restricciones únicas de `Certificado.inscripcion_id` y `Notificacion.evento_id` hacen que un evento repetido no genere duplicados.

Diagrama de secuencia

Figura 4

Secuencia para completar un curso y obtener el certificado



Nota. Elaboración propia.

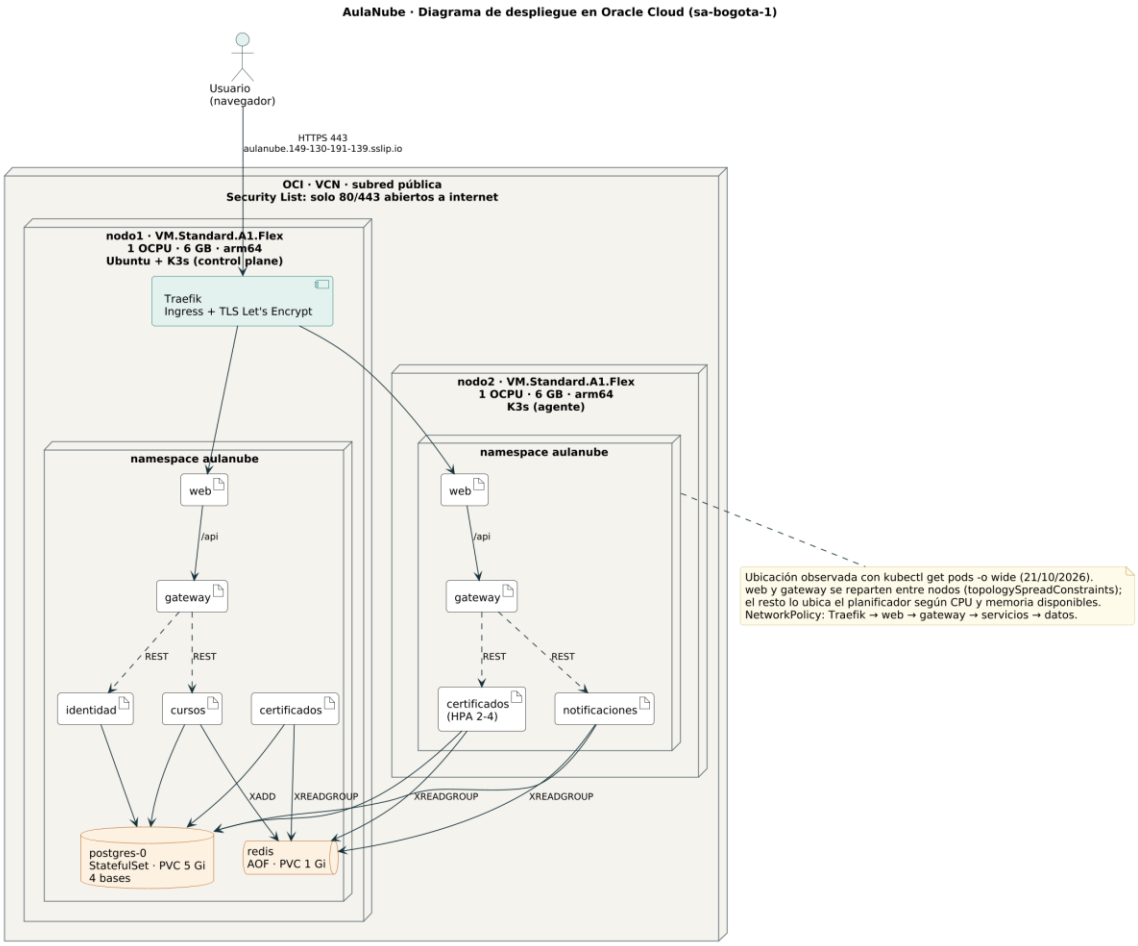
Al marcar la última lección, el servicio de cursos guarda el avance y el evento en la misma transacción y responde de inmediato. En segundo plano, un hilo publica el evento en el flujo `aulanube.cursos`. Los grupos de certificados y de notificaciones lo reciben en paralelo; el trabajador genera el PDF y publica `certificado.emitido`, que notificaciones convierte en el aviso para el estudiante. Este escenario resuelve los problemas del caso: el usuario no espera

(RNF02), el pico de una campaña queda en la cola (RNF01) y, si Redis falla, el evento permanece en la base hasta poder publicarse (RNF04).

Diagrama de despliegue

Figura 5

Despliegue en Oracle Cloud Infrastructure, región sa-bogota-1



Nota. Elaboración propia a partir de los manifiestos de Kubernetes (infra/k8s) y de la ubicación de los pods observada el 21 de octubre de 2026.

Dos instancias VM.Standard.A1.Flex de 1 OCPU y 6 GB forman un clúster K3s dentro de la capa gratuita de Oracle (s. f.). Traefik termina TLS con un certificado de Let's Encrypt. La web y el gateway tienen una réplica en cada nodo; certificados escala de dos a cuatro réplicas según CPU; PostgreSQL y Redis usan volúmenes persistentes, y las políticas de red solo

permiten el camino web, gateway, servicios y datos. El diagrama muestra así cómo se atienden la disponibilidad, la escalabilidad, la seguridad y el costo.

Selección y justificación de patrones de diseño

Los patrones se seleccionaron a partir de los escenarios de calidad y del catálogo de patrones para la nube de Microsoft (s. f.) y Richardson (2018). Blas et al. (2019) señalan que estos patrones deben modelarse y verificarse como parte de la arquitectura; por eso cada uno aparece en los diagramas anteriores y se comprueba en el prototipo. La Tabla 5 relaciona cada patrón con el problema que resuelve.

Tabla 5

Patrones de diseño seleccionados

Patrón	Aplicación en AulaNube	RNF
API Gateway y Gateway Offloading	Única entrada: valida el token, descarta las cabeceras de identidad del cliente y reenvía la del token, asigna un identificador de correlación.	05, 10
Database per Service	Una base y un usuario por servicio; cada usuario solo puede conectarse a la suya.	07, 08
Publisher/Subscriber	Un flujo de Redis Streams por publicador y un grupo de consumidores por servicio suscrito.	07
Transactional Outbox	El evento se guarda en la misma transacción que el cambio y un hilo lo publica después.	04
Queue-Based Load Leveling	La campaña masiva responde 202 al instante; la cola absorbe el pico.	01
Competing Consumers	Varias réplicas comparten la cola; las de una réplica caída se reclaman con XAUTOCLAIM.	01, 04
Circuit Breaker y Retry	Tras tres fallos el circuito se abre 15 s y responde de inmediato; las lecturas se reintentan una vez.	03
Rate Limiting	Cubeta de fichas de 20 solicitudes por segundo por empresa.	06
Health Endpoint Monitoring	Ruta /salud en cada servicio, sondas de Kubernetes y panel de estado.	03, 10
Repository	RepositorioCursos aplica el filtro por empresa en un solo lugar (Fowler, 2002).	05

Nota. Elaboración propia. La columna RNF indica el escenario de la Tabla 3 que atiende cada patrón.

El patrón Circuit Breaker responde al problema del correo que bloqueaba la finalización del curso: una integración lenta consume conexiones hasta tumbar al llamador, y el interruptor convierte ese fallo lento en uno rápido (Nygard, 2018). La bandeja de salida (Transactional

Outbox) resuelve que guardar en la base y publicar en la cola no pueden ser atómicos: el problema pasa a ser «publicar al menos una vez», que se cubre con consumidores idempotentes (Richardson, 2018). La nivelación de carga con consumidores en competencia separa la tasa de llegada de la tasa de proceso, de modo que la capacidad se ajusta con réplicas sin cambiar código. Se descartaron Saga, porque ningún flujo exige deshacer pasos en varios servicios, y un service mesh, cuyo consumo excede los nodos de 1 OCPU.

Prototipo funcional en la nube

Tecnologías y despliegue

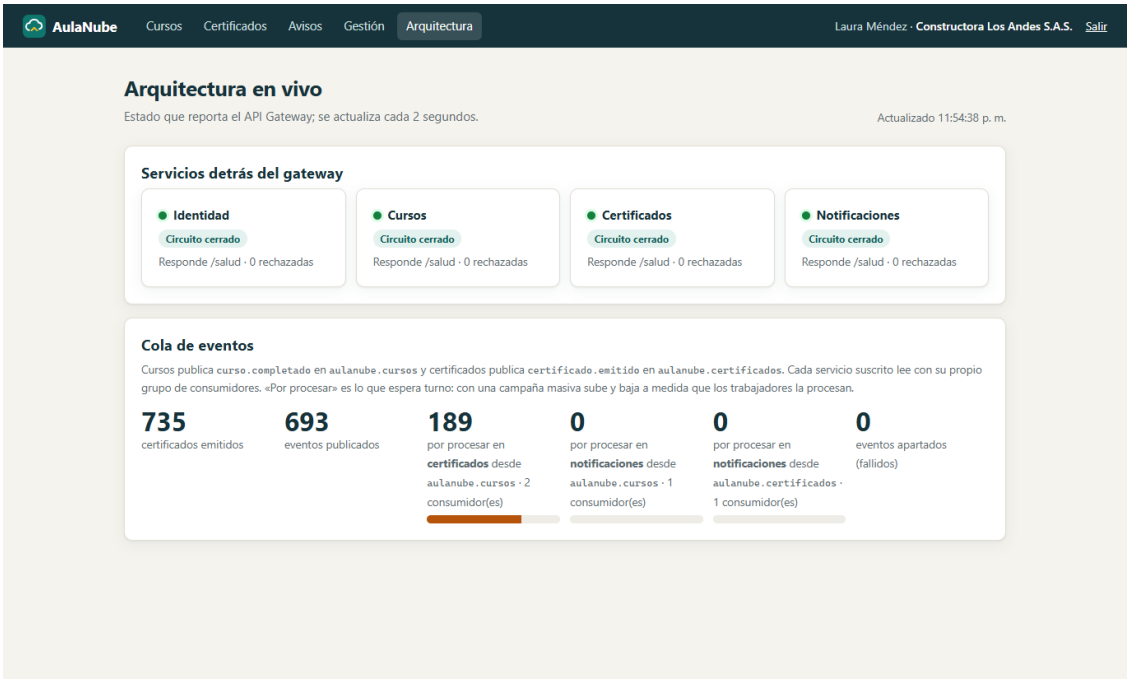
Los servicios están escritos en Python 3.12 con FastAPI y SQLAlchemy; la interfaz es una aplicación web sin dependencias servida por nginx; los datos residen en PostgreSQL 16 y los eventos en Redis 7.4. En local, el sistema completo se ejecuta con Docker Compose. En la nube, las imágenes se construyen para arm64 en el primer nodo, se importan en ambos y se aplican cinco manifiestos de Kubernetes; el script `scripts/desplegar.sh` permite actualizar un solo servicio sin detener los demás (RNF08). Las 28 pruebas automatizadas pasan tanto en x86 como en los nodos arm64, y una prueba de humo recorre el flujo completo contra el despliegue.

Evidencias de los patrones

Para comprobar la nivelación de carga se lanzó una campaña de 200 participantes. El servicio respondió en menos de 300 ms y la Figura 6 muestra la cola del grupo de certificados mientras dos réplicas la procesaban, a unos 6,5 certificados por segundo, sin afectar al resto de la plataforma.

Figura 6

Panel de arquitectura durante una campaña de 200 participantes

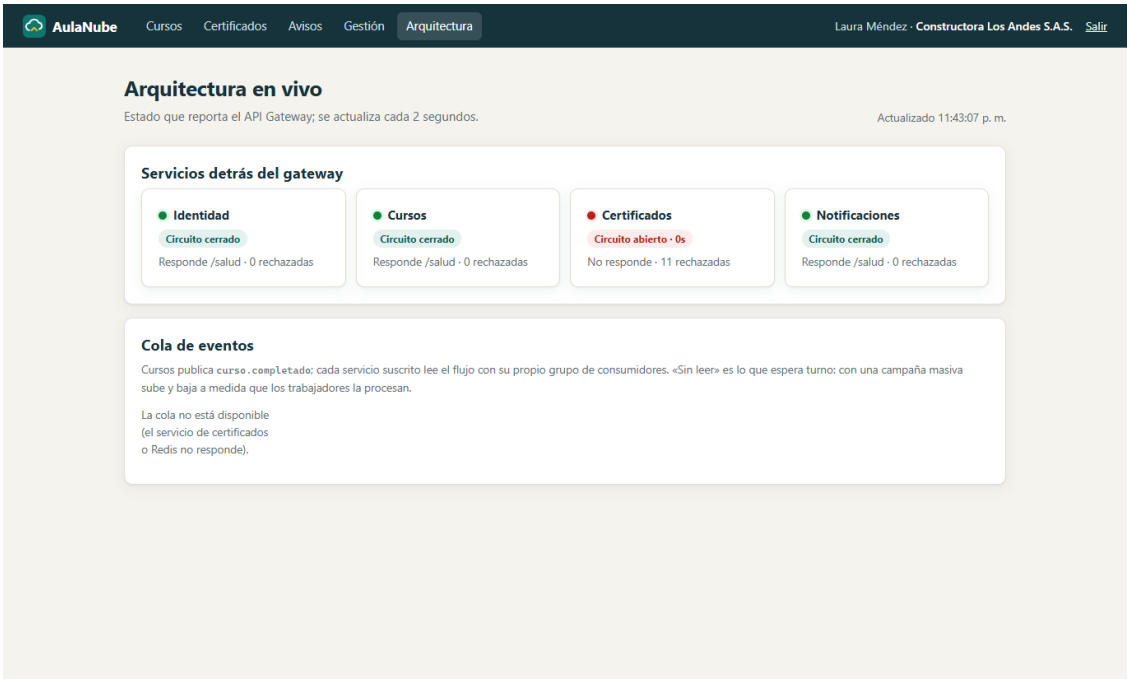


Nota. Captura del 21 de octubre de 2026 en el despliegue de Oracle Cloud Infrastructure.

Para comprobar el aislamiento de fallos se escaló el servicio de certificados a cero réplicas. Tras tres fallos, el gateway respondió con el circuito abierto en unos 110 ms, sin esperar al servicio caído, mientras el catálogo de cursos seguía respondiendo (Figura 7). Con el servicio apagado, una estudiante completó un curso: el evento quedó pendiente en Redis y el certificado se emitió nueve segundos después de volver a escalar el servicio, sin intervención manual.

Figura 7

Circuito del servicio de certificados abierto



Nota. Captura del 21 de octubre de 2026. Los demás servicios permanecen disponibles.

Tabla 6

Resultados medidos frente a los escenarios de calidad

RNF	Medida esperada	Resultado medido
RNF01	Respuesta a una campaña masiva < 1 s, sin errores	202 en 195–300 ms; 200 certificados emitidos sin fallidos
RNF02	Respuesta < 500 ms; certificado < 10 s	Finalización en unos 150 ms; certificado en 1–2 s
RNF03	Respuesta < 300 ms con un servicio caído	110–150 ms con el circuito abierto; cursos sin afectación
RNF04	Ningún certificado perdido	Evento retenido y procesado al recuperar el servicio
RNF05	Aislamiento entre empresas y servicios	Verificado con pruebas y con las políticas de red del clúster
RNF09	Capa gratuita	Dos nodos de 1 OCPU y 6 GB dentro de la franquicia Ampere A1

Nota. Mediciones del 21 de octubre de 2026 sobre el despliegue en la nube. Las limitaciones conocidas son que el límite de solicitudes y el estado de los circuitos viven en la memoria de cada réplica del gateway y que PostgreSQL y Redis tienen una sola réplica.

Conclusiones

Los problemas de CapacitaPro eran de arquitectura y no de código. Un servidor por grupo de clientes y un flujo síncrono hacían que cualquier pico o falla se propagara; al cambiar la estructura, esos síntomas desaparecieron sin optimizar la lógica de negocio, lo que confirma que las decisiones estructurales tempranas determinan la calidad del sistema (Bass et al., 2012).

Expresar los requisitos no funcionales como escenarios medibles permitió elegir los patrones con criterio y comprobarlos: cada patrón del prototipo atiende un escenario concreto y tiene una medición que lo respalda. A la vez, la cohesión, el acoplamiento y el ocultamiento de información dejaron de ser conceptos y se volvieron decisiones verificables, como una base por servicio y eventos con contratos pequeños.

La asincronía tiene costos que deben hacerse explícitos: consistencia eventual, entrega «al menos una vez» y consumidores idempotentes. Por último, una arquitectura en la nube no exige un presupuesto alto: el prototipo completo opera en la capa gratuita de Oracle con TLS, escalado automático y segmentación de red. Como trabajo futuro se propone trasladar los PDF a almacenamiento de objetos, usar una base de datos administrada con réplica, compartir el estado del gateway e integrar un proveedor de correo real.

Referencias

Bass, L., Clements, P., & Kazman, R. (2012). *Software architecture in practice* (3.ª ed.). Addison-Wesley.

Blas, M. J., Leone, H. P., & Gonnet, S. M. (2019). Modelado y verificación de patrones de diseño de arquitectura de software para entornos de computación en la nube. En *Proceedings del Congreso Nacional de Ingeniería Informática/Sistemas de Información*.

Congreso de la República de Colombia. (2012, 17 de octubre). *Ley Estatutaria 1581 de 2012: Por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial n.º 48.587. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Microsoft. (s. f.). *Cloud design patterns*. Azure Architecture Center. Recuperado el 20 de octubre de 2026, de https://learn.microsoft.com/en-us/azure/architecture/patterns/

Nygard, M. T. (2018). *Release it! Design and deploy production-ready software* (2.ª ed.). Pragmatic Bookshelf.

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), version 2.5.1*. https://www.omg.org/spec/UML/2.5.1

Oracle. (s. f.). *Always Free resources*. Oracle Cloud Infrastructure Documentation. Recuperado el 20 de octubre de 2026, de https://docs.oracle.com/en-us/iaas/Content/FreeTier/freetier*topic-Always*Free_Resources.htm

Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM, 15*(12), 1053–1058. https://doi.org/10.1145/361598.361623

Red Hat. (2023). *¿Qué son los microservicios? Ventajas de la arquitectura de microservicios*. https://www.redhat.com/es/topics/microservices/what-are-microservices

Redwerk. (2025). *Arquitectura monolítica frente a la de microservicios para .NET*. https://redwerk.es/blog/arquitectura-monolitica-vs-microservicios/

Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning.

Tecnológica del Oriente. (2026). *Arquitectura de software: Principios, patrones y evaluación. Unidad 1: Principios de arquitectura de software* [Material del curso Arquitectura de Software]. Institución de Educación Superior Tecnológica del Oriente.